PROJECT REPORT: HOSPITAL DEPARTMENT CONNECTIVITY AND SHORTEST ROUTE SYSTEM

Implementation Using Breadth-First Search (BFS) and Dijkstra's Algorithm in C++

Student Name	**Vaishali Rathod**	**Roll Number**	**26013**
Branch	**ENTC (Electronics & Telecommunication)**	**Subject**	Data Structures and Algorithms (BTL-3: Apply)
College	**Amrutvahini College of Engineering, Sangamner**		

1. PROBLEM STATEMENT

Original Problem Statement (BTL-3: Apply): Develop a C++ program to represent hospital departments as vertices and corridors as weighted edges. Apply Breadth-First Search (BFS) from the emergency department to display reachable units and apply Dijkstra's algorithm to calculate the shortest route and distance to a selected department.

In a multi-specialty hospital, emergency response teams, stretchers, and medical staff frequently transfer patients and critical supplies between specialized departments such as the Emergency Department, Radiology (X-Ray/CT), Intensive Care Unit (ICU), Operation Theatre, Pathology Lab, and Pharmacy. Representing the hospital layout as a mathematical graph where each medical unit is a vertex and each connecting corridor is an undirected weighted edge (measured in meters) enables systematic reachability verification and optimal path calculation.

This project implements an adjacency-list graph in C++ that models six core hospital departments connected by eight weighted corridors. Starting at the Emergency Department (vertex 0), the program executes Breadth-First Search (BFS) to list all reachable hospital units in level-by-level proximity order and applies Dijkstra's single-source shortest-path algorithm with a min-priority queue to compute the exact shortest corridor path and total distance in meters to any selected destination unit.

2. OBJECTIVES

- **Graph Representation of Hospital Floor Plan:** Model hospital medical units as graph vertices (0 to 5) and physical walkways/corridors as undirected weighted edges stored inside an adjacency list (`vector<vector<Edge>>`).
- **Reachability Verification via BFS:** Implement Breadth-First Search starting from the Emergency Department using a FIFO queue to discover and display all reachable medical departments in level order.
- **Optimal Route Computation via Dijkstra's Algorithm:** Apply Dijkstra's greedy relaxation algorithm using a binary min-heap priority queue to determine the minimum-distance transfer route from the Emergency Department to a user-selected target department.
- **Path Reconstruction & Distance Reporting:** Maintain predecessor pointers (`parent[]`) during edge relaxation so the complete sequence of intermediate departments and total meter distance can be displayed clearly.
- **Menu-Driven Console Interface:** Provide a continuous, validated menu loop that lets users inspect the corridor network, run BFS traversal, query shortest paths to multiple destinations, and exit

cleanly.

3. SYSTEM ARCHITECTURE AND COMPLEXITY SUMMARY

The architecture consists of an Edge structure (storing target vertex `to` and corridor length `weight`) and a `HospitalGraph` class that encapsulates department names, the adjacency list, BFS traversal, and Dijkstra's shortest-path solver. Because hospital floor networks are sparse graphs where each unit connects directly to only a few neighboring corridors, an adjacency list requires only **O(V + E)** memory rather than an **O(V^2)** adjacency matrix.

Function	Operation Performed	Time Complexity	Auxiliary Space
`addCorridor(u, v, w)`	Inserts undirected weighted corridor between departments u and v	O(1)	O(1) per edge
`displayGraph()`	Iterates through all vertices and prints connected corridors with weights	O(V + E)	O(1)
`bfsFromEmergency(0)`	Level-order traversal from Emergency Department using FIFO queue	O(V + E)	O(V)
`shortestRouteDijkstra()`	Computes shortest path & distance via min-priority queue and `parent[]`	O((V + E) log V)	O(V)

4. ALGORITHM

Step 1 — Start and Initialize Hospital Graph: Start the program, instantiate `HospitalGraph(6)`, assign department names for vertices `0` (Emergency Department), `1` (Radiology), `2` (ICU), `3` (Operation Theatre), `4` (Pathology Lab), and `5` (Pharmacy), and populate the eight undirected weighted corridor edges.

Step 2 — Display Corridor Adjacency List: For each vertex `i` from `0` to `V - 1`, print the department ID and name followed by every adjacent department and corridor distance in meters.

Step 3 — Apply Breadth-First Search (BFS) from Emergency Department: Allocate a boolean array `visited[V] = false` and an empty queue q. Mark `visited[0] = true` and enqueue vertex `0`. While q is non-empty, dequeue the front vertex `curr`, print its department name, and inspect all neighbors in `adjList[curr]`. For each unvisited neighbor, mark it visited and push it into q.

Step 4 — Apply Dijkstra's Shortest-Path Algorithm: Read the destination department ID `destDept`. Initialize `dist[0..V-1] = INF`, `parent[0..V-1] = -1`, and set `dist[0] = 0`. Insert `(0, 0)` into a min-priority queue pq. While pq is not empty, pop the pair `(d, u)` with minimum tentative distance. For each outgoing corridor `(u, to, weight)`, if `dist[u] + weight < dist[to]`, update `dist[to]`, set `parent[to] = u`, and push `(dist[to], to)` into pq.

Step 5 — Reconstruct Route and Repeat Menu: If `dist[destDept] == INF`, display *"No Route Available"*. Otherwise, trace `parent[]` backward from `destDept` to `0`, reverse the path vector, and display the complete department sequence and total distance in meters. Repeat the menu until option 4 (Exit) is selected.

5. SYSTEM FLOWCHART AND HOSPITAL GRAPH TOPOLOGY

The hospital corridor network modeled in the program links six critical care and diagnostic departments. Notice that although the Emergency Department has a direct 50-meter corridor to the ICU, traversing via Radiology (`30m + 15m = 45m`) provides a shorter path to the ICU and downstream units such as the Operation Theatre (`65m` total).

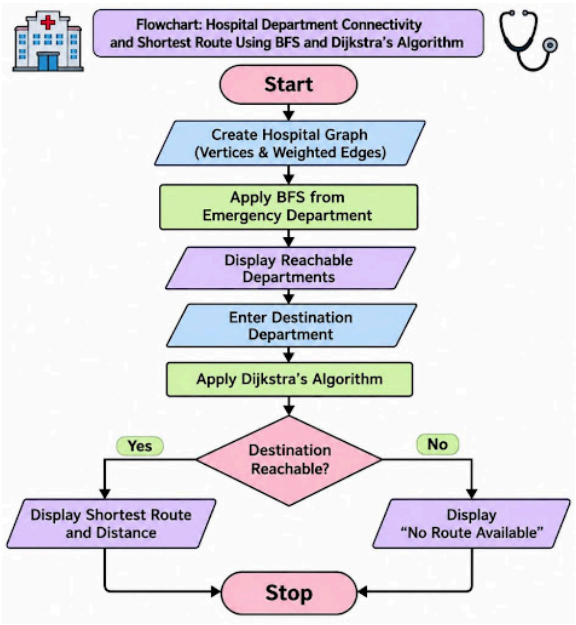


Figure 1: System Flowchart — Hospital Department Connectivity and Shortest Route Using BFS and Dijkstra's Algorithm

Vertex ID	Hospital Department Name	Connected Corridors (Neighbor & Distance in Meters)
[0]	Emergency Department (Source)	[1] Radiology (30m), [2] ICU (50m)
[1]	Radiology (X-Ray/CT)	[0] Emergency Dept (30m), [2] ICU (15m), [4] Pathology Lab (25m)
[2]	ICU	[0] Emergency Dept (50m), [1] Radiology (15m), [3] Operation Theatre (20m)
[3]	Operation Theatre	[2] ICU (20m), [4] Pathology Lab (35m), [5] Pharmacy (15m)
[4]	Pathology Lab	[1] Radiology (25m), [3] Operation Theatre (35m), [5] Pharmacy (20m)
[5]	Pharmacy	[4] Pathology Lab (20m), [3] Operation Theatre (15m)

6. SOURCE CODE IMPLEMENTATION (Hospital_Graph.cpp)

The complete C++17 implementation below defines the `Edge` struct, the `HospitalGraph` class with `bfsFromEmergency()` and `shortestRouteDijkstra()`, and the interactive menu loop in `main()`.

```
#include <iostream>
#include <vector>
#include <string>
#include <queue>
#include <limits>
#include <iomanip>
```

```
#include <algorithm>

using namespace std;

const int INF = numeric_limits<int>::max();

struct Edge {
    int to;
    int weight;
};

class HospitalGraph {
private:
    int numDepartments;
    vector<string> deptNames;
    vector<vector<Edge>> adjList;

public:
    HospitalGraph(int n) : numDepartments(n), deptNames(n), adjList(n) {}

    void setDepartmentName(int id, const string& name) {
        if (id >= 0 && id < numDepartments) {
            deptNames[id] = name;
        }
    }

    void addCorridor(int u, int v, int distance) {
        if (u >= 0 && u < numDepartments && v >= 0 && v < numDepartments && distance > 0) {
            adjList[u].push_back({v, distance});
            adjList[v].push_back({u, distance});
        }
    }

    void displayGraph() const {
        cout << "\n--- Hospital Department Corridor Network ---\n";
        for (int i = 0; i < numDepartments; ++i) {
            cout << "[" << i << "] " << left << setw(22) << deptNames[i] << " -> ";
            if (adjList[i].empty()) {
                cout << "No connected corridors";
            } else {
                for (size_t j = 0; j < adjList[i].size(); ++j) {
                    cout << deptNames[adjList[i][j].to] << " (" << adjList[i][j].weight << "m)";
                    if (j + 1 < adjList[i].size()) cout << ", ";
                }
            }
            cout << "\n";
        }
    }

    // Apply BFS from Emergency Department (vertex 0) to display reachable units
    void bfsFromEmergency(int startDept = 0) const {
        if (startDept < 0 || startDept >= numDepartments) {
            cout << "Invalid starting department ID.\n";
            return;
        }

        vector<bool> visited(numDepartments, false);
        queue<int> q;

        visited[startDept] = true;
        q.push(startDept);

        cout << "\n--- BFS Traversal from " << deptNames[startDept] << " ---\n";
        cout << "Reachable Hospital Units in Level Order:\n";

        int step = 1;
```

```
        while (!q.empty()) {
            int curr = q.front();
            q.pop();
            cout << "  " << step++ << ". [" << curr << "] " << deptNames[curr] << "\n";

            for (const auto& edge : adjList[curr]) {
                if (!visited[edge.to]) {
                    visited[edge.to] = true;
                    q.push(edge.to);
                }
            }
        }
    }

    // Apply Dijkstra's algorithm to calculate shortest route and distance
    void shortestRouteDijkstra(int startDept, int destDept) const {
        if (startDept < 0 || startDept >= numDepartments || destDept < 0 || destDept >=
numDepartments) {
            cout << "Invalid department ID selected.\n";
            return;
        }

        vector<int> dist(numDepartments, INF);
        vector<int> parent(numDepartments, -1);
        priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;

        dist[startDept] = 0;
        pq.push({0, startDept});

        while (!pq.empty()) {
            auto [d, u] = pq.top();
            pq.pop();

            if (d > dist[u]) continue;

            for (const auto& edge : adjList[u]) {
                if (dist[u] != INF && dist[u] + edge.weight < dist[edge.to]) {
                    dist[edge.to] = dist[u] + edge.weight;
                    parent[edge.to] = u;
                    pq.push({dist[edge.to], edge.to});
                }
            }
        }

        cout << "\n--- Dijkstra's Shortest Path Result ---\n";
        cout << "Source      : [" << startDept << "] " << deptNames[startDept] << "\n";
        cout << "Destination : [" << destDept << "] " << deptNames[destDept] << "\n";

        if (dist[destDept] == INF) {
            cout << "Status      : No Route Available (Destination is unreachable).\n";
            return;
        }

        vector<int> path;
        for (int v = destDept; v != -1; v = parent[v]) {
            path.push_back(v);
        }
        reverse(path.begin(), path.end());

        cout << "Shortest Route : ";
        for (size_t i = 0; i < path.size(); ++i) {
            cout << deptNames[path[i]];
            if (i + 1 < path.size()) cout << " -> ";
        }
        cout << "\nTotal Distance : " << dist[destDept] << " meters\n";
    }
```

```
    void listDepartments() const {
        cout << "\nAvailable Hospital Departments:\n";
        for (int i = 0; i < numDepartments; ++i) {
            cout << "   " << i << " : " << deptNames[i] << "\n";
        }
    }
};

int main() {
    HospitalGraph hospital(6);

    hospital.setDepartmentName(0, "Emergency Department");
    hospital.setDepartmentName(1, "Radiology (X-Ray/CT)");
    hospital.setDepartmentName(2, "ICU");
    hospital.setDepartmentName(3, "Operation Theatre");
    hospital.setDepartmentName(4, "Pathology Lab");
    hospital.setDepartmentName(5, "Pharmacy");

    // Preloaded hospital corridors with distances in meters
    hospital.addCorridor(0, 1, 30);
    hospital.addCorridor(0, 2, 50);
    hospital.addCorridor(1, 2, 15);
    hospital.addCorridor(1, 4, 25);
    hospital.addCorridor(2, 3, 20);
    hospital.addCorridor(4, 3, 35);
    hospital.addCorridor(4, 5, 20);
    hospital.addCorridor(3, 5, 15);

    int choice, dest;
    do {
        cout << "\n============================================\n";
        cout << "   HOSPITAL DEPARTMENT GRAPH NAVIGATION\n";
        cout << "============================================\n";
        cout << "1. Display Hospital Graph (Departments & Corridors)\n";
        cout << "2. Apply BFS from Emergency Department\n";
        cout << "3. Find Shortest Route using Dijkstra's Algorithm\n";
        cout << "4. Exit\n";
        cout << "Choice: ";
        if (!(cin >> choice)) {
            cout << "Invalid choice.\n";
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            continue;
        }

        switch (choice) {
            case 1:
                hospital.displayGraph();
                break;
            case 2:
                hospital.bfsFromEmergency(0);
                break;
            case 3:
                hospital.listDepartments();
                cout << "Enter Destination Department ID (0 to 5): ";
                cin >> dest;
                hospital.shortestRouteDijkstra(0, dest);
                break;
            case 4:
                cout << "Exiting...\n";
                break;
            default:
                cout << "Invalid choice.\n";
        }
    } while (choice != 4);
```

```
    return 0;
}
```

7. CONSOLE EXECUTION OUTPUT AND TESTING

During execution, selecting Option 1 displays the complete adjacency list of all 6 hospital departments and their corridor lengths. Selecting Option 2 performs BFS from the Emergency Department, visiting units in level order: Emergency Department → Radiology → ICU → Pathology Lab → Operation Theatre → Pharmacy. Selecting Option 3 runs Dijkstra's algorithm: for Destination 3 (Operation Theatre), it identifies the optimal **65-meter** route (`Emergency Department → Radiology → ICU → Operation Theatre`), and for Destination 5 (Pharmacy), it computes the **75-meter** route (`Emergency Department → Radiology → Pathology Lab → Pharmacy`).

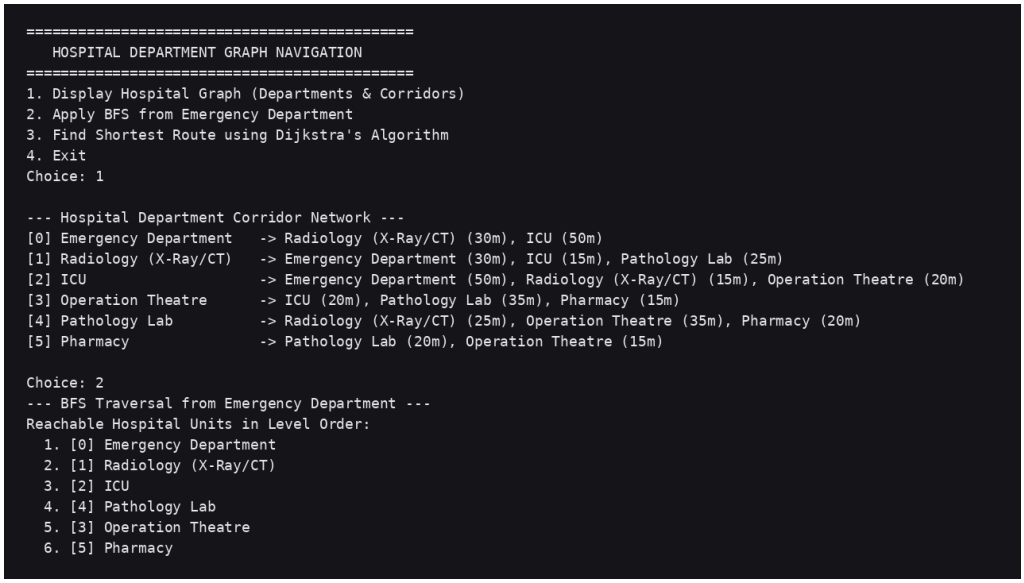


Figure 2: Console Execution Output — Graph Adjacency List Display and BFS Level-Order Traversal

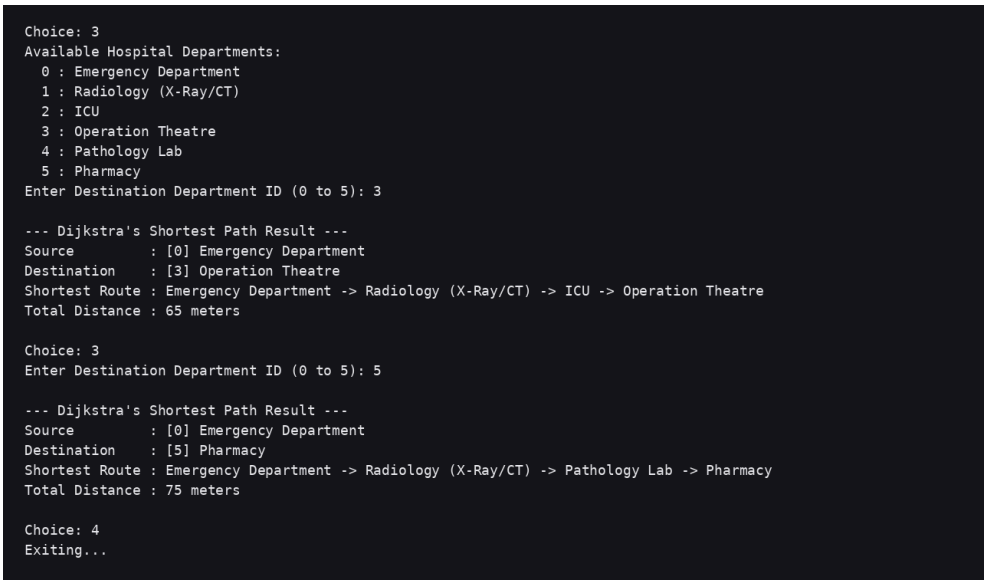


Figure 3: Console Execution Output — Dijkstra's Shortest Route to Operation Theatre (65m) and Pharmacy (75m)

Test ID	Test Scenario	User Input / Action	Verified Output
TC-01	Display Hospital Corridor Network	Select Option 1	Lists vertices [0] to [5] with exact bidirectional corridor distances in meters.

TC-02	BFS Traversal from Emergency Dept	Select Option 2	Outputs all 6 reachable units in level order starting from [0] Emergency Department.
TC-03	Shortest Route to Operation Theatre	Option 3 → Destination ID 3	Selects `0 → 1 → 2 → 3` (65m) instead of longer direct ICU path `0 → 2 → 3` (70m).
TC-04	Shortest Route to Pharmacy	Option 3 → Destination ID 5	Computes `Emergency → Radiology → Pathology Lab → Pharmacy` with total distance 75m.
TC-05	Invalid Department / Menu Input	Enter out-of-range ID or non-integer	Reports invalid input, clears input stream, and preserves menu loop stability.

8. CONCLUSION AND FUTURE SCOPE

The Hospital Department Connectivity and Shortest Route System demonstrates how graph data structures and fundamental graph algorithms solve practical healthcare logistics problems. Representing departments as vertices and corridors as weighted edges in an adjacency list achieves optimal **O(V + E)** space efficiency. Breadth-First Search (BFS) verifies hospital-wide connectivity from the Emergency Department in **O(V + E)** time, while Dijkstra's algorithm accurately identifies the minimum-distance patient transfer route in **O((V + E) log V)** time.

In particular, Dijkstra's relaxation mechanism proves essential when multiple paths exist with different cumulative weights—for example, routing an emergency stretcher to the ICU and Operation Theatre via Radiology saves 5 meters compared to taking the direct Emergency-to-ICU corridor. Future enhancements can incorporate dynamic graph updates (adding new hospital wings or corridors at runtime), before-and-after route comparisons during corridor maintenance, and automated alerts for isolated or disconnected hospital wards.

Compilation Command: `g++ -std=c++17 -Wall -Wextra Hospital_Graph.cpp -o Hospital_Graph`